

整体思路

一、核心概念映射：从 EUREKA 到 AI Agent

在开始写代码前，我们需要把 EUREKA 的组件映射到 Agent 的标准结构（Perception, Brain, Action, Memory）上：

1. **Agent (The Brain):** 这里的角色是“高级奖励工程师”。它负责阅读代码、理解物理规律、编写 Python 代码以及分析实验日志。
2. **Tools (The Hands):** Agent 需要手（工具）来验证它的想法。
 - **代码写入工具：** 将生成的字符串保存为 `.py` 文件。
 - **仿真执行工具：** 也就是我们之前说的“评估脚本”，负责跑 RL 训练并返回 `Fitness Score` 和 `TensorBoard Logs`。
3. **Memory (The Context):**
 - **短期记忆：** 当前这一轮生成的代码及其表现。
 - **长期记忆 (Evolutionary Archive)：** 过去所有迭代中最好的奖励函数代码（Parent Code）及其反思记录。

二、详细实现流程：构建 "Evolutionary Reward Agent"

我们不使用普通的对话 Agent，而是构建一个 **Loop-Based Agent**（循环式智能体）。我建议使用 **LangChain (LangGraph)** 或 **MetaGPT** 这种框架来管理状态。

以下是具体的**五步**工作流：

1. 初始化阶段 (Initialization)

Agent 启动，加载环境上下文。

- **输入：** 多无人机环境源代码（`Raw Environment Code`）、任务描述（例如：“实现多机协同围捕，避免碰撞”）。
- **Agent 动作：** 阅读代码，分析 Observation Space（观测空间）和 Action Space（动作空间），建立对物理世界的初步认知。

2. 探索阶段 (Generation / Mutation) —— 这是 Agent 的思考过程

Agent 需要生成一批候选的奖励函数。在 EUREKA 中，这是一个进化的过程。

- **Prompt 构建：** 系统自动将“父本代码（上一轮最好的代码）” + “上一轮的反思 (Reflection)” + “环境代码” 拼接成 Prompt。
- **Agent 推理：**
 - 思考：“上一轮虽然成功率高，但是无人机之间容易相撞。我需要在奖励函数中增加一个基于相对距离的排斥场 (Repulsive Potential Field)。”
 - 行动：编写 \$K\$ 个不同的 `compute_reward()` 函数变体。

3. 执行阶段 (Tool Execution) —— 这是 Agent 的行动过程

Agent 调用工具与仿真环境交互。这不是 LLM 在跑，而是 LLM 指挥 CPU/GPU 在跑。

- **工具调用：** `Executor_Tool(code_list)`。
- **底层操作（脚本自动完成）：**
 1. 语法检查 (Syntax Check)：过滤掉编译报错的代码。
 2. 并行分发：将通过检查的代码分发到不同的 GPU 进程中。
 3. 训练监控：运行轻量级训练（例如 500-1000 iterations）。
 4. 数据回收：收集每个代码对应的 `Success Rate` 和 `Reward Component Statistics`（如：距离惩罚项的均值、方差）。

4. 感知与反思阶段 (Perception & Reflection) —— 这是 Agent 的进化关键

这是最体现“智能”的一步。Agent 不是只看一个分数，而是像人类研究员一样“读图表”。

- **输入：** 训练日志（文本化）。
- **Agent 动作：**
 - 阅读反馈：“代码 A 的距离惩罚项一直为 0，说明阈值设置太高了，根本没触发。”
 - 生成反思 (Reward Reflection)：Agent 用自然语言写一段总结：“当前奖励函数在引导‘接近目标’方面表现良好，但在‘队形保持’上失效。下一轮需要调大队形保持项的权重系数，并引入非线性惩罚。”

5. 记忆更新 (Memory Update / Evolution)

- **操作：** 将本轮表现最好的代码和对应的反思存入长期记忆。

- **循环：** 回到第 2 步，将本轮的最佳代码作为下一轮的“父本”，开始新一轮变异。

三、技术架构图 (Mental Model)

如果要把这个 Agent 写出来，代码结构大概是这样的：

Python

```
class RewardDesignAgent:
    def __init__(self, llm_model="gpt-4"):
        self.llm = llm_model
        self.memory = EvolutionaryMemory() # 存储进化树
        self.tools = [SimulationTool()] # 挂载仿真工具

    def step(self, current_best_code, reflection):
        # 1. 思考与生成 (Thought & Action)
        prompt = self.construct_prompt(current_best_code, reflection)
        candidate_codes = self.llm.generate(prompt, n=16) # 生成16个样本

        # 2. 调用工具 (Tool Use)
        results = self.tools[0].run_parallel(candidate_codes)

        # 3. 结果分析 (Observation)
        best_result = max(results, key=lambda x: x['fitness'])

        # 4. 自我反思 (Reflection)
        new_reflection = self.llm.analyze(best_result['logs'])

        return best_result['code'], new_reflection

# 主循环 (The Workflow)
agent = RewardDesignAgent()
code = initial_code
reflection = ""

for iter in range(5):
    print(f"--- Evolution Generation {iter} ---")
    code, reflection = agent.step(code, reflection)
    agent.memory.save(code, reflection)
```

一、仿真工具的底层逻辑与实现细节

你的理解完全正确，但为了让这个工具更健壮，我们需要明确“插入”和“收集”的具体操作。

1. 逻辑流程图

这个仿真工具本质上是一个 **黑盒函数 (Black-box Function)**：输入是文本（代码字符串），输出是数值（性能指标）和文本（日志分析）。

2. “插入”代码的具体实现

通常我们不建议用复杂的 Python `exec()` 动态执行字符串（虽然可以，但不好调试）。EUREKA 的官方实现或者是工程上的最佳实践通常是 **文件覆盖法**：

1. **占位符模板**：在你的环境代码（例如 `env_poker.py`）中，预留一个 `compute_reward` 函数的位置，或者将奖励计算逻辑独立成一个文件 `reward_function.py`。
2. **热更新**：主控 Agent 拿到 LLM 生成的代码字符串后，直接将其写入覆盖 `reward_function.py` 文件。
3. **重载**：训练脚本（`train.py`）在启动时，`import reward_function`。

3. 日志在哪里加？（关键）

是的，日志代码必须加在训练脚本（`train.py`）或环境封装（`env_wrapper.py`）中。

大模型之所以能“反思”，全靠 EUREKA 提出的 **Reward Reflection** 机制。这意味着你不能只告诉它“成功了”或“失败了”，你必须告诉它**“奖励函数里的每一项（Component）到底发生了什么”**。

二、多无人系统围捕：你需要定义的“高质量日志”

这是你毕设创新的关键点。EUREKA 论文中强调，必须追踪奖励函数中各个分量的**标量值 (Scalar Values)**。

针对你的**多无人系统围捕 (Multi-UAV Encirclement)** 任务，我建议你设计以下三个维度的日志：

1. 奖励分量诊断 (Reward Component Breakdown)

这是给 LLM 调参用的。如果某一项一直是 0，说明逻辑没触发；如果某一项数值巨大，说明它主导了训练。

- `dist_reward_mean`：距离奖励的均值（是否在靠近？）。

- `collision_penalty_mean` : 碰撞惩罚的均值（是否过于激进？）。
- `formation_reward_mean` : 队形保持奖励的均值（是否学会了合作？）。
- `energy_cost_mean` : 能耗惩罚。

2. 任务性能指标（Task Performance Metrics）

这是判断“行不行”的硬指标。

- `success_rate` : 围捕成功率（例如：100次里成功几次）。
- `capture_time` : 平均围捕耗时（效率）。
- `safety_score` : 无碰撞步数占比（安全性）。

3. 协同行为洞察（Collaborative Behavioral Insights）—— 创新点

这是让 LLM 学会“多智能体协作”的关键线索。

- `encirclement_angle` (**围捕角**): 所有无人机相对于目标的角度分布方差。方差越小，说明包围圈越均匀。
- `min_agent_distance` : 智能体之间的最近距离（用于检测是否扎堆）。
- `escape_rate` : 目标成功逃脱包围圈的次数。

代码示例（伪代码）：

Python

```
# 在 train.py 的 step 循环中
next_obs, reward, done, info = env.step(actions)

# --- 你的日志收集逻辑 ---
# 假设 reward 是由 components 字典组成的（这是 EUREKA 的要求）
# components = {'dist': 0.1, 'collision': -10.0, ...}
current_reward_components = env.get_last_reward_components()

# 存入 buffer
logger.log(current_reward_components)
logger.log({'formation_quality': calculate_formation_quality(global_state)})
```

三、LangGraph 是什么？为什么你需要它？

你现在的思维还在“写一个 Python 脚本跑循环”的阶段，而 **LangGraph** 是帮你把这个循环变得结构化、可视化的框架。

1. 通俗解释

想象一下，普通的 Python 脚本像是一条直线，从头走到尾。而 Agent 的工作流往往是循环的（**Cyclic**）、有状态的（**Stateful**）。

LangGraph 是 LangChain 团队推出的一个库，专门用来定义图（**Graph**）结构的应用。它把你的流程看作一个个节点（**Nodes**）和边（**Edges**）。

- **Node（节点）**：做事情的地方。比如“生成代码节点”、“运行仿真节点”、“写反思节点”。
- **Edge（边）**：决定下一步去哪。比如“如果仿真报错 → 回到生成代码节点修 bug”；“如果成功率 > 90% → 结束”。

2. 为什么要用它？

对于你的毕设，使用 LangGraph 有以下好处：

1. **状态管理（Persistence）**：它会自动帮你保存“当前最好的代码”、“历史反思记录”、“尝试次数”等状态，你不需要自己写一堆全局变量来维护。
2. **容错循环（Human-in-the-loop）**：如果大模型写出的代码一直报错，LangGraph 可以让你定义一个逻辑：连续报错3次就暂停，发消息给你（人类），让你手动修一下，然后继续跑。这对于调试毕设非常有帮助。
3. **可视化调试**：它能生成一张流程图，让你清晰地看到 Agent 此时卡在哪个环节（是卡在生成，还是卡在仿真）。

具体实现步骤

假设本轮迭代 LLM 生成了 $K=4$ 个奖励函数代码，并行流程如下：

1. 代码生成与隔离 (Code Generation & Isolation)

不要覆盖同一个文件。你的主控 Agent 应该为每一个生成的代码分配一个**唯一的 ID**。

- **动作**：将 4 份代码分别保存为：
 - `experiments/candidate_0/reward_function.py`
 - `experiments/candidate_1/reward_function.py`
 - `experiments/candidate_2/reward_function.py`
 - `experiments/candidate_3/reward_function.py`

- **注意：** 你的 `train.py` 也需要复制到这些文件夹里，或者让 `train.py` 能够动态加载指定路径的奖励函数。最简单的工程做法是：**为每个候选者创建一个独立的文件夹（Sandbox），把所有运行环境代码都软链接（Symbolic Link）或者复制进去，只替换 `reward_function.py`。**

2. 编写并行分发脚本 (The Dispatcher)

这是你必须要写的一个 Python 脚本（比如叫 `launcher.py`），用来同时“点火”启动这 4 个训练任务。

我们利用 Python 的 `multiprocessing` 库或 `subprocess` 模块来实现。

代码示例（你可以直接参考）：

Python

```
import subprocess
import time
import os
from multiprocessing import Pool

# 定义单个任务的运行逻辑
def run_single_training(candidate_id):
    # 1. 确定工作目录
    work_dir = f"./experiments/candidate_{candidate_id}"

    # 2. 组装命令： python train.py --log_dir ./logs
    # 这里的关键是：让训练脚本知道日志存哪里，别存混了
    cmd = [
        "python", "train.py",
        "--reward_path", "reward_function.py", # 让脚本加载当前目录的奖励
        "--log_dir", "logs"
    ]

    print(f"🚀 [Candidate {candidate_id}] 开始训练...")

    # 3. 启动子进程 (使用 subprocess)
    # capture_output=True 可以捕获打印的日志，避免屏幕上乱成一团
    try:
        result = subprocess.run(
            cmd,
```

```

    cwd=work_dir,          # 【核心】切换到独立的沙盒目录运行
    capture_output=True,
    text=True,
    timeout=1200           # 设置超时，防止死循环
)

# 4. 训练结束，简单解析一下输出，或者直接去读 logs 文件
if result.returncode == 0:
    print(f"✅ [Candidate {candidate_id}] 训练完成")
    return {"id": candidate_id, "status": "success"}
else:
    print(f"❌ [Candidate {candidate_id}] 训练报错:\n{result.stderr[-200:]}")
    return {"id": candidate_id, "status": "error"}

except subprocess.TimeoutExpired:
    print(f"🕒 [Candidate {candidate_id}] 训练超时")
    return {"id": candidate_id, "status": "timeout"}

# 主程序
if __name__ == "__main__":
    candidate_ids = [0, 1, 2, 3] # 假设有4个代码

    # 5. 并行池：同时跑 4 个进程
    # 注意：这里 process 数量受限于你的 CPU 核数和显存大小
    # 如果是多无人机环境（MPE/SMAC），CPU 可能是瓶颈
    with Pool(processes=4) as pool:
        results = pool.map(run_single_training, candidate_ids)

    print("所有训练结束，汇总结果：", results)

```

3. 资源分配与显存管理 (GPU VRAM Management)

这是最容易踩坑的地方。

- **问题：** 假设每个训练进程要占 2G 显存，你的一块 3090 (24G) 最多能跑 10-12 个。如果你强行开 16 个进程，会报 **OOM (Out Of Memory)** 错误。
- **解决方案：**

- **排队机制：** 上面的 `Pool(processes=N)` 中的 `N` 就是并发数。如果有 16 个任务，但 `N=4`，Python 会自动调度：先跑前 4 个，跑完一个补一个，直到全部跑完。
- **指定 GPU：** 如果你有通过多卡，可以在 `cmd` 环境变量里指定 `CUDA_VISIBLE_DEVICES`。例如任务 0-3 跑在 GPU0，任务 4-7 跑在 GPU1。

总结一下流程

1. **LLM 生成阶段：** 生成 4 段代码字符串。
2. **沙盒构建阶段：** 你的脚本自动创建 `candidate_0` 到 `candidate_3` 四个文件夹，把对应的代码写进去，并把环境基座代码复制进去。
3. **并行启动阶段：** 运行 `launcher.py`，它利用 CPU 多核特性，同时唤起 4 个独立的 Python 解释器。
 - 进程 1 跑在 `candidate_0` 文件夹里。
 - 进程 2 跑在 `candidate_1` 文件夹里。
 - ...
4. **互不干扰：** 因为工作目录（CWD）不同，它们生成的日志文件（tensorboard logs, txt logs）会自然地保存在各自的文件夹里，不会覆盖。
5. **结果回收：** `launcher.py` 等待所有子进程结束后，去每个文件夹里读取日志，打包发给 LLM。

毕设小贴士：

由于你是做多无人机（Multi-UAV），这类环境（如 MPE）通常是 CPU 密集型的。并行跑多个训练非常吃 **CPU**。

- 如果你的电脑是 8 核的，建议并行数设为 4-6，留点资源给系统，否则电脑会卡死。
- 如果训练太慢，可以考虑缩减评估时的 `max_episodes`（比如训练没必要完全收敛，只要能看出趋势即可提前终止，EUREKA 论文中经常只跑部分步数来快速验证）。

第一阶段：环境接口与标准化 (The Foundation)

目标： 让大模型“看懂”物理世界，让代码能“插入”数学模型。

1. 环境解耦：

- 将你的多无人机仿真环境（基于MPE或SMAC改进）中的奖励计算逻辑剥离出来，封装成独立的 `reward_function.py` 或特定的接口函数。
- 确保该函数输入包含：`global_state`（用于协同计算）、`local_obs`（用于单机计算）、

`actions` 等关键变量。

2. 上下文提取 (Context Extraction):

- 编写脚本自动提取环境源代码的“骨架”：即包含 `reset()`, `step()`, 以及观测空间 (Observation Space) 定义的代码片段。

关键点： 这是 LLM 理解任务物理约束（如无人机最大速度、传感器范围）的唯一来源。

第二阶段：大模型核心循环 (The Agent Brain)

目标： 构建一个具备“编码”和“反思”能力的智能体。

1. 初始生成 (Zero-Shot Generation):

◦

Input： 环境代码骨架 + 任务描述（如“围捕目标且避免碰撞”）。

- **Process：** 调用 GPT-4，要求其生成 K 个（如 4 个）不同的奖励函数代码。

◦

Format： 要求输出标准 Python 代码，并返回一个包含各分量 (Component) 的字典，而不仅仅是总分。

2. 进化机制 (Evolutionary Search):

- 在后续迭代中，将上一轮表现最好的代码 (Elite) 作为“父本”。

◦

Mutation Prompt： 结合“父本代码”和“训练反馈”，要求 LLM 进行修改（如：改变权重系数、增加新的惩罚项、改变函数形式）。

第三阶段：并行训练沙盒 (The Execution Engine)

目标： 高效验证 LLM 的“脑洞”，避免资源冲突。

1. 沙盒隔离 (Sandboxing):

- 编写 `launcher.py` 主控脚本。
- 为 LLM 生成的每份代码创建一个独立文件夹（如 `exp_iter1_sample0`）。
- 将环境基座代码、训练脚本（`train.py`）和生成的 `reward_function.py` 放入该文件夹。

2. 并行分发 (Parallel Dispatching):

- 利用 `multiprocessing` 或 `subprocess` 同时启动 K 个训练进程。
- **资源调度：** 根据 CPU 核数和 GPU 显存，控制并发数量，防止 OOM（内存溢出）。

3. 轻量化验证：

◦ 不需要跑满整个收敛过程（例如原本需要 100 万步，验证时跑 10-20 万步即可），只看趋势和中期表现，以节省时间。

第四阶段：反馈与闭环 (The Reflection Loop)

目标：让数据说话，指导 LLM 优化。

1. 日志增强 (Log Instrumentation):

- 在 `train.py` 中埋点，专门记录**奖励分量统计值**（如：距离项均值、碰撞项方差）。
- 增加**任务特定指标**：对于围捕任务，记录“围捕角方差”、“逃逸率”、“编队紧密度”。

2. 反思生成 (Reward Reflection):

- 将上述统计数据转化为文本描述（例如：“碰撞惩罚项数值过大，导致智能体原地不动”）。
- 将这段文本反馈给 LLM，触发下一轮的进化。

3. 循环控制:

- 设定迭代次数 N （例如 5 轮）。
- 循环结束后，输出历史最佳奖励函数，作为最终 MARL 训练的定稿方案。

总结图示 (Mental Model)

这个流程图（参考论文中的 Figure 2）在你脑海中应该简化为：

环境代码 → LLM → K 份代码 → 并行沙盒训练 → 日志分析 → LLM (带着反思) → 新一轮代码

导师的最后叮嘱

同学，这个设计方案已经非常完善，达到了硕士甚至博士开题的高标准。它不仅解决了多智能体强化学习中****奖励设计难（Reward Engineering）的问题，还引入了大模型推理（Reasoning）****来辅助决策。

接下来的行动建议:

1. **先跑通“哑巴”流程**：不要接 LLM，先手动写两个不同的 `reward_function.py`，用你的 `launcher.py` 脚本试着并行跑起来，确保文件夹隔离和日志回收没有 Bug。
2. **再接“大脑”**：流程跑通后，再把 OpenAI API 接进来，替换掉你的手动操作。